Goldfarb–Idnani Revisited: Invariants, Certificates, and the Limits of Guessing

Thomas Schmelzer[1] and Martin Stoll[2]

[1]Jebel Quant Research, Abu Dhabi
[2]Faculty of Mathematics, TU Chemnitz, Germany

Abstract

Guessing a whole active set at once and repairing it from the signs that come back, as primal–dual active set and block principal pivoting do, terminates finitely for the bound-constrained quadratic program. The system solved on a candidate set is then a principal submatrix of G^{-1}, and inherits that matrix's P-matrix property. Our first result is that the general constraint class costs exactly that. For $C^\top x \geq b$ the working-set system is $C_{\mathcal{A}}^\top G^{-1} C_{\mathcal{A}}$, positive definite only when $C_{\mathcal{A}}$ has full column rank, which is a property of the *guess* and not of the data. The P-matrix property is lost, and with it the guarantee. The obstruction is structural, not a missing anti-cycling rule.

That exposure is easy to mistake for a technicality, and we show it is not. Over six hundred random instances across four constraint families it never occurs, which is why it can be overlooked. Duplicating columns of C, as any programmatically assembled model may, raises it to 93%, and the certified fraction then collapses from 100% to nothing.

To make the argument precise we first give a self-contained derivation of the Goldfarb–Idnani dual method, organised around a single matrix J satisfying $J^\top G J = I$ and $J^\top A = [R; 0]$. Every quantity the iteration computes follows from those two lines and admits a representation-free closed form, so any (J, R) meeting the invariants produces the same iterate. The objective increment along a step of length t is exactly $t(t/2 + u_*)\, z^\top G z$. And when the primal cannot move and no multiplier can be reduced, the vector r the solver already holds is a Farkas certificate, so the infeasibility verdict is a proof.

What makes the unguaranteed methods usable is that strict convexity makes the KKT conditions sufficient, so every candidate can be certified. On long-only portfolio data the active set reaches 442 of $494 + 1$ constraints, against the small sets produced by the synthetic families common in this literature.

1 Introduction

The strictly convex quadratic program

$$\min_{x \in \mathbb{R}^n} \ \tfrac{1}{2} x^\top G x - a^\top x \qquad \text{subject to} \qquad C^\top x \geq b, \tag{1}$$

with $G \in \mathbb{R}^{n \times n}$ symmetric positive definite, $C \in \mathbb{R}^{n \times m}$ and the first m_{eq} of the m constraints understood as equalities, is solved a great many times a day. Mean-variance portfolio selection [13] is exactly (1); so is every step of a sequential quadratic programming method and every horizon of a linear model predictive controller. For dense instances of small to moderate size, with n from a handful to a few thousand, the method of choice has for forty years been the dual active-set algorithm of Goldfarb and Idnani [7, 8], whose distinguishing property is that it needs no phase-one

problem: the unconstrained minimiser $G^{-1}a$ is dual feasible for free, so the iteration can start there and drive primal infeasibility to zero.

This paper derives that method in the form the `cvx-quadprog` package [16] implements it, together with the two extensions that package adds. Its aim is to state every identity the solver depends on, derive it, and name the degenerate cases. Section 9 then checks each one against the code that ships, which is a different question from whether it is true. Every empirical claim made anywhere below is measured there; none is quoted from elsewhere.

1.1 Why the dual method

A primal active-set method maintains a feasible iterate and works towards stationarity. It must therefore begin at a feasible point, and finding one is a linear program in its own right, the phase-one problem. The dual method reverses the roles. It maintains stationarity and the sign conditions on the multipliers throughout, and works towards feasibility. Its starting point is free, because with no constraint active the stationarity condition reads $Gx = a$ and the multiplier vector is empty, so $x = G^{-1}a$ satisfies everything the method asks of an iterate. One Cholesky factorisation and one triangular solve replace the phase-one problem entirely.

The price is that the iterate is infeasible until the last step, so the method cannot be stopped early for an approximate answer. In exchange it terminates at an exactly feasible point of the active-set lattice rather than approaching one asymptotically as an interior-point method does, and it warm-starts almost perfectly, which is why parametric solvers are built on active-set methods [15]. Section 8 exploits that.

1.2 What has happened to the method since 1983

The method has not stood still, and three lines of work bear on what is claimed below. Its restriction to $G \succ 0$ was lifted by Boland [4], who generalises the dual method to the semidefinite case with a proof of finite termination; the strict convexity kept here is a choice of scope, since the factorisation of Section 3 needs G^{-1}. Finite termination under nondegeneracy is not special to this method either: Forsgren, Gill and Wong [6] prove it for paired primal and dual active-set methods in a more general setting.

The modern successor is not a reimplementation but a different factorisation. Arnström, Bemporad and Axehill [2] carry the same dual walk on recursive $LDL^\top$ updates rather than the (J, R) pair used here, handle semidefinite G by outer proximal-point iterations, and ship it as library-free C for embedded model-predictive control; it is benchmarked against this solver in Section 9.2, being the fair comparison: same family, different linear algebra. Arnström and Axehill [1] additionally compute the *exact* worst-case iteration count of active-set methods on parametric QP families. That is stronger than anything proved here, and different in kind: theirs is computational and per-family, enumerating the subproblem sequences an algorithm visits over a parameter set, where Proposition 5.2 is general but conditional on nondegeneracy.

1.3 Contributions

1. **Why finite termination does not survive the general constraint class, which is the one claim here we believe to be new.** For bound constraints the working-set system is a principal submatrix of G^{-1}, hence positive definite whatever the guess, and that P-matrix property is the hypothesis under which block principal pivoting terminates finitely [12, 14, 17]. For $C^\top x \geq b$ it is lost, structurally, not for want of an anti-cycling rule (Section 7), and the

consequence is measured, not asserted: over 600 attempts on well-posed random families the failure never occurs, and repeating columns of C raises it to 93% (Section 9.1).

2. **A derivation organised around two invariants.** The solver carries one matrix J, defined by $J^\top G J = I$ and $J^\top A = [R; 0]$, and every quantity the iteration computes follows from those two lines (Section 3): the columns of J are a G-orthonormal basis of $\mathbb{R}^n$, split so that the trailing block spans the working set's feasible subspace and $J_2 J_2^\top$ is the reduced inverse Hessian.

3. **The iterates do not depend on the representation.** Invariants (10)–(11) do not determine (J, R); the freedom is exactly $(J_1 S, J_2 V, SR)$ for a sign matrix S and an orthogonal V. Both quantities the iteration consumes are invariant under all of it (Proposition 3.2), and exactly so in IEEE arithmetic in the sign case. The implementation is therefore free to pick its orthogonal reduction for speed alone.

4. **An exact objective increment.** A step of length t changes the objective by $t(t/2 + u_*)\, z^\top G z$ (Proposition 4.3), so the value is carried and not re-evaluated, and the increment is positive in both step directions.

5. **The infeasibility verdict is a certificate.** When the primal cannot move and no multiplier can be reduced, the vector r the solver already holds exhibits infeasibility in the sense of Farkas (Proposition 5.3). The method does not fail to converge on an infeasible problem; it proves the problem infeasible.

6. **Linear dependence is unreachable.** The step rules make the working-set normals full column rank for free, so no rank test is needed anywhere (Proposition 6.1), and guessing forfeits it.

7. **The cost of a warm start, corrected.** Recovery from cached factors is $O(n^2)$ and not $O(nk)$, the n^2 residing entirely in $x_u = J J^\top a$ (Section 8). The distinction is invisible on the family one would naturally test, where k grows with n; Section 8 separates the two by holding k fixed, and separates the arithmetic from the interpreter dispatch that dominates below a few hundred variables.

1.4 Outline

Sections 2 to 6 derive the method: the optimality conditions and the working-set subproblem, the invariants, the iteration and its termination, and the two factorisation updates. Sections 7 and 8 treat the two departures from the classical method, and Sections 9 and 10 the measurements, first against the claims above and then on real data.

1.5 Notation

Vectors are columns. For an index set $\mathcal{A} \subseteq \{1, \ldots, m\}$ of size k we write $A = C_{\cdot\mathcal{A}} \in \mathbb{R}^{n \times k}$ for the matrix of the corresponding constraint normals, $b_\mathcal{A}$ for the corresponding right-hand sides, and c_i for the i-th column of C. Throughout, $\mathcal{A}$ is the solver's *working set*: the constraints it is currently holding as equalities. At a solution the working set is a subset of the constraints active there, and the two coincide except in the degenerate case treated in Remark 5.1. The entering constraint's normal is n_* and its right-hand side b_*. We reserve u for multipliers of working-set constraints, u_* for the entering constraint's own multiplier, and λ for a full length-m multiplier vector with zeros off the working set.

2 The problem and its optimality conditions

Write the program (1) as

$$\min_{x\in\mathbb{R}^n} \; f(x) := \tfrac{1}{2}x^\top G x - a^\top x \qquad \text{subject to} \qquad \begin{cases} c_i^\top x = b_i, & i \le m_{\mathrm{eq}}, \\ c_i^\top x \ge b_i, & i > m_{\mathrm{eq}}, \end{cases} \tag{2}$$

with $G \succ 0$. Two conventions differ from the textbook ones and are inherited from the reference implementation's signature. The linear term is *subtracted*, so that the unconstrained minimiser is $G^{-1}a$ rather than $-G^{-1}a$; and the constraints are held column-wise in C, so that $C^\top x \ge b$ rather than $Cx \le b$. Neither affects anything below beyond signs.

2.1 Existence, uniqueness, sufficiency

Let $\Omega = \{x : c_i^\top x = b_i \ (i \le m_{\mathrm{eq}}),\ c_i^\top x \ge b_i \ (i > m_{\mathrm{eq}})\}$ denote the feasible set, a closed convex polyhedron.

Since $G \succ 0$ the objective is strictly convex and coercive, so (2) has exactly one solution whenever $\Omega \neq \emptyset$; see [15, Ch. 16]. What the method needs beyond that is the converse of the usual necessary conditions, and we prove it because every certificate in this paper rests on it.

The Lagrangian of (2) is $L(x,\lambda) = \tfrac{1}{2}x^\top G x - a^\top x - \lambda^\top (C^\top x - b)$, and the Karush–Kuhn–Tucker conditions are

$$Gx - a = C\lambda, \tag{3}$$

$$c_i^\top x = b_i \quad (i \le m_{\mathrm{eq}}), \qquad c_i^\top x \ge b_i \quad (i > m_{\mathrm{eq}}), \tag{4}$$

$$\lambda_i \ge 0 \quad (i > m_{\mathrm{eq}}), \tag{5}$$

$$\lambda_i \,(c_i^\top x - b_i) = 0 \quad (i > m_{\mathrm{eq}}). \tag{6}$$

Multipliers of equality constraints are unrestricted in sign, which is the one asymmetry the algorithm has to carry through every subsequent formula.

Proposition 2.1 (Sufficiency)**.** *If (x,λ) satisfies (3)–(6) then x is the unique solution of (2).*

Proof. Let $\tilde{x} \in \Omega$. By strict convexity, $f(\tilde{x}) \ge f(x) + (Gx - a)^\top(\tilde{x} - x)$, with equality only if $\tilde{x} = x$. Substituting (3),

$$(Gx-a)^\top(\tilde{x}-x) = \lambda^\top\!\left(C^\top\tilde{x} - C^\top x\right) = \sum_i \lambda_i\left[(c_i^\top\tilde{x} - b_i) - (c_i^\top x - b_i)\right] = \sum_i \lambda_i (c_i^\top \tilde{x} - b_i),$$

the last step by (6) for the inequalities and by $c_i^\top x = b_i$ for the equalities. Each surviving term is non-negative: for $i \le m_{\mathrm{eq}}$ the bracket vanishes, and for $i > m_{\mathrm{eq}}$ both factors are non-negative by (4) and (5). Hence $f(\tilde{x}) \ge f(x)$ with equality only at $\tilde{x} = x$. □

Proposition 2.1 is the licence for everything in Sections 7 and 8. A point equipped with multipliers passing the four conditions is *the* answer, so a method that produces candidates may be as unprincipled as one likes provided each candidate is tested. Sufficiency is what makes an unguaranteed method safe rather than usually right.

2.2 The working-set subproblem

Fix a working set $\mathcal{A}$ of size k with normals $A = C_{\cdot\mathcal{A}}$ and suppose A has full column rank k. The *working-set subproblem* is

$$\min_x \ \tfrac{1}{2}x^\top Gx - a^\top x \qquad \text{subject to} \qquad A^\top x = b_{\mathcal{A}}, \tag{7}$$

with every constraint in $\mathcal{A}$ held as an equality regardless of its type in the original program, and every constraint outside $\mathcal{A}$ ignored.

Proposition 2.2 (Solution of the subproblem)**.** *Problem* (7) *has the unique solution and multipliers*

$$u = (A^\top G^{-1}A)^{-1}(b_{\mathcal{A}} - A^\top x_u), \qquad x = x_u + G^{-1}A\,u, \qquad x_u := G^{-1}a. \tag{8}$$

Proof. Stationarity for (7) reads $Gx - a = Au$, so $x = G^{-1}a + G^{-1}Au = x_u + G^{-1}Au$. Substituting into $A^\top x = b_{\mathcal{A}}$ gives $A^\top x_u + (A^\top G^{-1}A)u = b_{\mathcal{A}}$. The matrix $A^\top G^{-1}A$ is positive definite because $G^{-1} \succ 0$ and A has full column rank, hence invertible, which yields (8); the point so constructed is the unique minimiser by strict convexity of f on the affine feasible set of (7). □

We call $A^\top G^{-1}A$ the *dual Hessian* of the working set. It appears under three different names in what follows: as the matrix whose Cholesky factor the solver carries (Section 3), as the coefficient matrix of the fast path's linear system (Section 7), and as the Gram matrix of the least-squares problem the dual direction solves (Lemma 4.1). Its invertibility is equivalent to A having full column rank, and that single fact is where every rank condition in this paper comes from.

The pair (x, u) of (8), extended by zeros off the working set, satisfies stationarity (3) and complementarity (6) by construction, since the working-set constraints have zero slack and the rest have zero multiplier. What it need not satisfy is the sign condition (5) on $\mathcal{A}$'s inequalities, or primal feasibility (4) off $\mathcal{A}$. The dual method maintains the first and works towards the second.

Remark 2.1 (Cold start)**.** *For $\mathcal{A} = \emptyset$ the subproblem is unconstrained and* (8) *degenerates to $x = x_u = G^{-1}a$ with an empty multiplier vector. The sign condition holds vacuously, so this state satisfies the method's invariant with no work beyond one Cholesky factorisation. A dual method's phase-one problem is nothing more. The objective there is $f(x_u) = \tfrac{1}{2}x_u^\top Gx_u - a^\top x_u = -\tfrac{1}{2}a^\top x_u$, which the implementation uses as the initial value of the running objective.*

2.3 The dual problem

The following explains the method's name, and also why the objective value can be tracked as a running total and why it increases.

Proposition 2.3 (Dual function)**.** *The Lagrangian dual of* (2) *is the concave program* max $\psi(\lambda)$ *over $\lambda_i \geq 0$ $(i > m_{\text{eq}})$, where*

$$\psi(\lambda) = -\tfrac{1}{2}(a + C\lambda)^\top G^{-1}(a + C\lambda) + b^\top\lambda. \tag{9}$$

If (x, λ) satisfies stationarity (3) *and complementarity* (6)*, then $\psi(\lambda) = f(x)$.*

Proof. Minimising $L(\cdot, \lambda)$ gives $x(\lambda) = G^{-1}(a + C\lambda)$ and, writing $s = a + C\lambda$, $\psi(\lambda) = \tfrac{1}{2}s^\top G^{-1}s - s^\top G^{-1}s + b^\top\lambda$, which is (9). For the second claim (3) gives $s = Gx$, and (6) with $c_i^\top x = b_i$ on the equalities gives $b^\top\lambda = (Gx - a)^\top x$; adding the two yields $f(x)$. □

So along any sequence of states of the form (8) the tracked value is at once the primal objective at the subproblem minimiser and the dual objective at the current multipliers. The strict increase of Proposition 4.3 is therefore progress towards optimality rather than bookkeeping, and it gives the sandwich $\psi(\lambda) \leq f(x^\star) \leq f(\tilde{x})$: the method approaches the optimal value from below, the mirror image of a primal active-set method.

3 The matrix the solver carries

Proposition 2.2 gives the subproblem solution in closed form, and one could implement the method by evaluating (8) afresh at every iteration. That costs a $k \times k$ factorisation of the dual Hessian per step, $O(nk^2 + k^3)$, and over the $O(n)$ steps a dual method takes it comes to $O(n^4)$. The whole point of the Goldfarb–Idnani formulation is to replace it with $O(n^2)$ per step, and it does so by carrying a single matrix through the iteration and updating it.

3.1 Two invariants

Let $G = R_G^\top R_G$ be the Cholesky factorisation with R_G upper triangular, and set $J_0 := R_G^{-1}$, again upper triangular. Then $J_0 J_0^\top = R_G^{-1} R_G^{-\top} = G^{-1}$ and $J_0^\top G J_0 = I$. The solver's state is a matrix $J \in \mathbb{R}^{n \times n}$ and an upper triangular $R \in \mathbb{R}^{k \times k}$ satisfying

$$J^\top G J = I_n, \tag{10}$$

$$J^\top A = \begin{bmatrix} R \\ 0 \end{bmatrix}. \tag{11}$$

Everything the iteration computes is a consequence of these two lines, so read them slowly. Equation (10) is equivalent to $J J^\top = G^{-1}$ with J nonsingular, and says that the columns of J form a basis of $\mathbb{R}^n$ orthonormal in the inner product $\langle v, w \rangle_G = v^\top G w$. Equation (11) says that in that basis the working-set normals are triangular: partitioning

$$J = \begin{bmatrix} J_1 & J_2 \end{bmatrix}, \qquad J_1 \in \mathbb{R}^{n \times k}, \quad J_2 \in \mathbb{R}^{n \times (n-k)}, \tag{12}$$

the lower block of (11) reads $J_2^\top A = 0$.

At the cold start $\mathcal{A} = \emptyset$, so (11) is vacuous and $J = J_0$ serves. The updates of Section 6 maintain both invariants by post-multiplying J with orthogonal matrices, which preserves (10) exactly because $J^\top G J = I$ is invariant under $J \mapsto JW$ for $W^\top W = I$, while restoring the triangular shape of (11).

Proposition 3.1 (What the blocks mean)**.** *Suppose* (10)–(11) *hold with* R *nonsingular. Then*

(i) $R^\top R = A^\top G^{-1} A$*, so* R *is a Cholesky factor of the dual Hessian, unique up to the signs of its rows;*

(ii) $\operatorname{range}(J_2) = \operatorname{null}(A^\top)$*, and the columns of* J_2 *are a* G*-orthonormal basis of it;*

(iii) $J_1 = G^{-1} A R^{-1}$ *and* $\operatorname{range}(J_1) = \operatorname{range}(G^{-1} A)$*, the* G*-orthogonal complement of* $\operatorname{null}(A^\top)$*;*

(iv) $J_1 J_1^\top = G^{-1} A (A^\top G^{-1} A)^{-1} A^\top G^{-1}$ *and consequently*

$$J_2 J_2^\top = G^{-1} - G^{-1} A (A^\top G^{-1} A)^{-1} A^\top G^{-1}. \tag{13}$$

Proof. (i) $A^\top G^{-1} A = A^\top J J^\top A = R^\top R$ by (11), and two upper triangular matrices with the same Gram matrix differ by a left factor $\operatorname{diag}(\pm 1)$.

(ii) $J_2^\top A = 0$ places $\operatorname{range}(J_2) \subseteq \operatorname{null}(A^\top)$, with equality by dimension since J is nonsingular and A has full column rank; the basis is G-orthonormal because $J_2^\top G J_2 = I_{n-k}$ is the trailing block of (10).

(iii) From (10), $J^{-1} = J^\top G$ and hence $J^{-\top} = GJ$. Inverting (11), $A = J^{-\top}[R; 0] = GJ[R; 0] = GJ_1R$, so $J_1 = G^{-1}AR^{-1}$, whose range is that of $G^{-1}A$ since R is invertible. That range is G-orthogonal to $\mathrm{null}(A^\top)$: for $v \in \mathrm{null}(A^\top)$, $\langle G^{-1}Aw, v\rangle_G = w^\top A^\top v = 0$.

(iv) Substituting $J_1 = G^{-1}AR^{-1}$ and $R^\top R = A^\top G^{-1}A$,

$$J_1J_1^\top = G^{-1}AR^{-1}R^{-\top}A^\top G^{-1} = G^{-1}A(R^\top R)^{-1}A^\top G^{-1} = G^{-1}A(A^\top G^{-1}A)^{-1}A^\top G^{-1},$$

and (13) follows from $J_1J_1^\top + J_2J_2^\top = JJ^\top = G^{-1}$. □

The matrix in (13) is the *reduced inverse Hessian* of the working set: it inverts G on the feasible subspace $\mathrm{null}(A^\top)$ and annihilates the complement. Carrying J therefore means carrying that object in factored form, at no cost beyond the $n \times n$ array, which turns the search directions of Section 4 into single matrix–vector products.

Remark 3.1 (Why fold the two factorisations together)**.** *An alternative state is R_G together with an explicit orthogonal factor Q of the QR factorisation of $R_G^{-\top}A$. That holds the same information, J being J_0Q, and stores no more numbers. But every use of J in Section 4 would become a triangular solve followed by an orthogonal transformation, two operations where the folded form has one, and the solve is the shape that does not reach a tuned kernel. The folded form has J lose its triangularity after the first update, which is exactly the trade: n^2 storage and $2n^2$ flops per matrix–vector product, in exchange for the products being* `gemv` *at every iteration whatever the working set does.*

3.2 The representation is not unique, and does not need to be

Invariants (10)–(11) do not determine J and R. By Proposition 3.1, R is pinned only up to a left factor $S = \mathrm{diag}(\pm 1)$, whereupon $J_1 = G^{-1}AR^{-1}$ follows; and J_2 may be any G-orthonormal basis of $\mathrm{null}(A^\top)$, so it is pinned only up to a right factor V with $V^\top V = I_{n-k}$. The freedom is therefore exactly

$$(J_1, J_2, R) \longmapsto (J_1S,\ J_2V,\ SR), \qquad S = \mathrm{diag}(\pm 1), \quad V^\top V = I_{n-k}. \tag{14}$$

Different orthogonal reductions realise different points of that family: a chain of Givens rotations and a single Householder reflection annihilating the same vector produce factors differing in sign, and a differently ordered sequence of insertions and deletions produces a different J_2 outright. The following says this does not matter, so the implementation may choose its reduction on grounds of speed alone.

Proposition 3.2 (Invariance of the iterates)**.** *Let (J, R) and $(\hat{J}, \hat{R})$ both satisfy (10)–(11) for the same G and A. Then for every $n_* \in \mathbb{R}^n$ the quantities*

$$d = J^\top n_*, \qquad z = J_2d_2, \qquad r = R^{-1}d_1 \tag{15}$$

with $d = (d_1, d_2)$ split at k, satisfy $\hat{z} = z$ and $\hat{r} = r$. Moreover, if the two representations are related by (14) with $V = I$, the equalities hold exactly in IEEE 754 arithmetic.

Proof. By Proposition 3.1(i) and $A^\top J = [R^\top\ 0]$ we have $A^\top G^{-1}n_* = A^\top JJ^\top n_* = R^\top d_1$, so

$$r = R^{-1}d_1 = R^{-1}R^{-\top}R^\top d_1 = (R^\top R)^{-1}A^\top G^{-1}n_* = \left(A^\top G^{-1}A\right)^{-1}A^\top G^{-1}n_*, \tag{16}$$

which mentions only A, G and n_*. Likewise $z = J_2J_2^\top n_*$, since $d_2 = J_2^\top n_*$, and by (13)

$$z = \left(G^{-1} - G^{-1}A\left(A^\top G^{-1}A\right)^{-1}A^\top G^{-1}\right)n_*, \tag{17}$$

again free of the representation. For the last claim, $\hat{R} = SR$ gives $\hat{d}_1 = Sd_1$ and $\hat{r} = (SR)^{-1}Sd_1 = R^{-1}S^{-1}Sd_1$, in which the only floating-point operations beyond those forming r are multiplications by ± 1; these are exact, so the computed $\hat{r}$ and r agree bit for bit. The same argument applies to $\hat{z} = \sum_{j>k}(s_j J_{\cdot j})(s_j d_j)$. □

Remark 3.2 (What the proposition does and does not settle)**.** *It settles that the substitution is exact in the sense that matters: the iteration's decisions (which constraint is most violated, whether the step is full or partial, which constraint is dropped) are functions of z and r alone, and those are functions of (G, A, n_*) alone. It does not settle that two implementations follow the same trajectory in finite precision, because the rounding* within *the reductions differs and the decisions are discontinuous functions of the computed values. That is an empirical question, and settling it properly needs two independent implementations to compare trajectories against each other. What is checkable against one, and is checked in the Reproducibility note, is the weaker statement: that the z and r the solver computes agree with the representation-free closed forms* (17) *and* (16) *to machine precision.*

4 One iteration

The state at the top of an outer iteration is a working set $\mathcal{A}$ of size k, the factors (J, R) satisfying (10)–(11), the subproblem solution x and multipliers u of Proposition 2.2, and the running objective $f(x)$. The invariant maintained throughout is

$$(x, u) \text{ solves the subproblem (7), and } \quad u_i \geq 0 \ \text{ for every inequality } i \in \mathcal{A}, \tag{18}$$

which by the discussion after Proposition 2.2 is stationarity, complementarity and dual feasibility together. Only primal feasibility is missing, and the iteration exists to obtain it.

4.1 Choosing the entering constraint

Compute the slacks $s_i = c_i^\top x - b_i$ for every i. If $s_i \geq 0$ for all inequalities and $s_i = 0$ for all equalities, then (4) holds, (18) supplies the rest, and x is the solution by Proposition 2.1. Otherwise a violated constraint is selected as the entering constraint. The rule is the most violated one, measured relative to the norm of its own normal:

$$v_i = \begin{cases} |s_i|, & i \leq m_{\text{eq}}, \\ -s_i, & i > m_{\text{eq}}, \end{cases} \qquad i_* \in \arg\max_i \ \frac{v_i}{\|c_i\|}, \qquad \text{terminating when } \max_i \frac{v_i}{\|c_i\|} \leq 0, \tag{19}$$

so that an equality counts as violated in either direction while an inequality counts only when its slack is negative. The scaling is not cosmetic.

Proposition 4.1 (Selection is scale invariant)**.** *Replacing (c_i, b_i) by $(\gamma c_i, \gamma b_i)$ for some $\gamma > 0$ leaves the feasible set, the solution, and the choice made by* (19) *unchanged.*

Proof. The constraint $\gamma c_i^\top x \geq \gamma b_i$ has the same solution set as $c_i^\top x \geq b_i$. Its slack scales as $s_i \mapsto \gamma s_i$ and its normal as $\|c_i\| \mapsto \gamma\|c_i\|$, so the ratio in (19) is unchanged, as are the other constraints' ratios. □

Remark 4.1 (The invariance is exact, and covers the whole walk)**.** *Proposition 4.1 claims only that the* choice *is unchanged. Measured against the implementation over three decades of γ either side of unity, the statement is stronger: the solver performs an identical number of additions and removals, so the whole trajectory is unchanged and not only its first step, and returns a minimiser agreeing to 3.3×10^{-16}.*

Without the division, a constraint written in basis points rather than in units would be selected first merely for having been written differently, and the solver's trajectory, though not its answer, would depend on the caller's choice of units. Note that the multiplier of the rescaled constraint scales as $\lambda_i \mapsto \lambda_i/\gamma$, so the sign conditions are unaffected too.

4.2 The two directions

Write $n_* = c_{i_*}$ and $b_* = b_{i_*}$, and form d, z and r as in (15). The following lemma is the computational core of the method; everything in the rest of this section is a corollary of it.

Lemma 4.1 (Properties of the directions)**.** *With d, z, r as in (15),*

(i) $A^\top z = 0$*: moving along z preserves the working-set equalities;*

(ii) $n_*^\top z = \|d_2\|^2 = z^\top G z \geq 0$*, with equality iff $z = 0$;*

(iii) $Gz = n_* - Ar$*;*

(iv) $z = 0$ *if and only if $n_* = Ar$, i.e. iff n_* lies in the span of the working-set normals, in which case r holds its coordinates there.*

Proof. (i) $A^\top z = A^\top J_2 d_2 = (J_2^\top A)^\top d_2 = 0$ by (11).

(ii) $n_*^\top z = n_*^\top J_2 d_2 = (J_2^\top n_*)^\top d_2 = d_2^\top d_2$. For the second equality, $z^\top G z = d_2^\top (J_2^\top G J_2) d_2 = d_2^\top d_2$ by the trailing block of (10). Since J_2 has full column rank, $z = 0$ iff $d_2 = 0$.

(iii) By (13), $Gz = G J_2 J_2^\top n_* = n_* - A(A^\top G^{-1} A)^{-1} A^\top G^{-1} n_*$, and the second term is Ar by (16).

(iv) If $d_2 = 0$ then $J^\top n_* = (d_1, 0) = [R; 0] R^{-1} d_1 = (J^\top A) r$, and J is nonsingular, so $n_* = Ar$. Conversely $n_* = Ar$ gives $Gz = n_* - Ar = 0$ by (iii), hence $z = 0$. □

Part (ii) identifies z as an ascent direction for the entering constraint's slack: the slack $n_*^\top x - b_*$ grows at rate $n_*^\top z > 0$ per unit step. Part (i) says the step costs nothing in working-set feasibility. Together with the closed forms (17) and (16) the reading is:

- z is the G-orthogonal projection of the unconstrained direction $G^{-1} n_*$ onto the working set's feasible subspace $\mathrm{null}(A^\top)$. Indeed $\Pi := J_2 J_2^\top G$ is idempotent with range $\mathrm{null}(A^\top)$ and is self-adjoint for $\langle \cdot, \cdot \rangle_G$, and $z = \Pi\, G^{-1} n_*$.
- r solves a least-squares problem in the G^{-1} metric. By (16),

$$r = \operatorname*{arg\,min}_{y \in \mathbb{R}^k} \|J^\top A y - J^\top n_*\|_2 = \operatorname*{arg\,min}_{y \in \mathbb{R}^k} (Ay - n_*)^\top G^{-1} (Ay - n_*), \tag{20}$$

  the best approximation of the entering normal by the working-set normals. Its residual, mapped through G^{-1}, is z: from Lemma 4.1(iii), $z = G^{-1}(n_* - Ar)$.

Remark 4.2 (Why not call a least-squares routine)**.** *Equation (20) is a genuine least-squares problem and it is tempting to hand it to a library. The temptation should be resisted: the factor R of its normal equations is already held, so (15) costs one triangular solve, where a library call would refactorise $J^\top A$ from scratch at $O(nk^2)$ every iteration. The same remark applies to (8): the closed form is the specification, not the implementation.*

4.3 The affine path

Fix the directions and consider moving along them. Let u_* denote the entering constraint's own multiplier, which starts at 0.

Proposition 4.2 (Dual feasibility along the path)**.** *For $t \in \mathbb{R}$ define*

$$x(t) = x + tz, \qquad u(t) = u - tr, \qquad u_*(t) = u_* + t. \tag{21}$$

Then for all t: $A^\top x(t) = b_{\mathcal{A}}$, and stationarity holds for the working set augmented by the entering constraint,

$$Gx(t) - a = A\,u(t) + u_*(t)\,n_*. \tag{22}$$

Proof. The first claim is Lemma 4.1(i). For the second, stationarity at $t = 0$ reads $Gx - a = Au + u_* n_*$, and by Lemma 4.1(iii),

$$Gx(t) - a = (Gx - a) + t\,Gz = Au + u_* n_* + t(n_* - Ar) = A(u - tr) + (u_* + t)n_*. \qquad \square$$

So the entire path is stationary, with the entering multiplier rising at unit rate and the working-set multipliers falling linearly along $-r$. Complementarity holds for every constraint except the entering one, whose slack is still nonzero while its multiplier is positive; that single violation is what the step is taken to remove. Two limits on t follow immediately.

The dual limit t_1. Dual feasibility requires $u_i(t) \geq 0$ for every inequality $i \in \mathcal{A}$. Since $u_i(t) = u_i - tr_i$ and $u_i \geq 0$, the binding indices are those with $r_i > 0$, and

$$t_1 = \min\Big\{ \frac{u_i}{r_i} \;:\; i \in \mathcal{A},\ i > m_{\text{eq}},\ r_i > 0 \Big\}, \qquad i_{\text{del}} = \text{an index attaining it}, \tag{23}$$

with $t_1 = +\infty$ when the set is empty. Equality constraints are excluded: their multipliers are unrestricted in sign, so no step in t can make them infeasible. This is the classical ratio test, and t_1 is the largest step that preserves (18).

The primal limit t_2. The entering constraint's slack is $n_*^\top x(t) - b_* = s_{i_*} + t\,n_*^\top z$ with $s_{i_*} < 0$, so it reaches zero at

$$t_2 = \frac{|s_{i_*}|}{n_*^\top z} = \frac{|s_{i_*}|}{\|d_2\|^2}, \tag{24}$$

finite and positive whenever $z \neq 0$, and $t_2 = +\infty$ when $z = 0$ by Lemma 4.1(ii), in which case the primal iterate cannot move at all and only the multipliers change.

The step. Take $t = \min(t_1, t_2)$. If $t_2 \leq t_1$ the step is *full*: the entering constraint now holds with equality, so it joins the working set with multiplier $u_* + t_2$ and the factorisation is updated (Section 6). The new state satisfies (18): stationarity and complementarity by Proposition 4.2 with the slack now zero, and the sign conditions because $t_2 \leq t_1$. If $t_1 < t_2$ the step is *partial*: the multiplier of i_{del} reaches zero first, that constraint leaves the working set, the factorisation is downdated, and z and r are recomputed, both having changed with J and R, keeping the same entering constraint and its accumulated u_*, and the loop repeats. If both limits are infinite the problem is infeasible; see Proposition 5.3.

Remark 4.3 (Equalities violated from above)**.** *An equality constraint may be violated in either direction. When $s_{i_*} > 0$ the slack must be* decreased*, so the step is taken along $-z$: formally t ranges over the negative reals, $t_2 = -|s_{i_*}|/n_*^\top z$, the ratio test in (23) runs against $-r$ rather than r, and the entering multiplier accumulates negatively. Nothing else changes; in particular Proposition 4.2 was stated for all $t \in \mathbb{R}$ precisely so that this case needs no separate treatment, and Proposition 4.3 below covers it too.*

4.4 The objective, in closed form

The running objective is not re-evaluated from x; it is advanced by an exact increment.

Proposition 4.3 (Objective increment)**.** *Along the path (21),*

$$f(x(t)) - f(x) = t\Big(\frac{t}{2} + u_*\Big)\, n_*^\top z, \tag{25}$$

which is strictly positive whenever $z \neq 0$ and $t \neq 0$ and t, u_* have the same sign (in particular whenever $t > 0$ and $u_* \geq 0$).

Proof. Expanding the quadratic, $f(x + tz) = f(x) + t\, z^\top(Gx - a) + \frac{1}{2}t^2\, z^\top G z$. For the linear term, stationarity at $t = 0$ and Lemma 4.1(i) give $z^\top(Gx - a) = z^\top A u + u_*\, z^\top n_* = u_*\, n_*^\top z$. For the quadratic term, Lemma 4.1(ii) gives $z^\top G z = n_*^\top z$. Hence $f(x + tz) - f(x) = t u_* n_*^\top z + \frac{1}{2}t^2 n_*^\top z$, which is (25). Positivity: $n_*^\top z > 0$ by Lemma 4.1(ii), and $t(t/2 + u_*) > 0$ when t and u_* share a sign and $t \neq 0$. □

The increment (25) is exact, so the objective can be carried as a running total at the cost of one multiplication per step instead of an $O(n^2)$ re-evaluation of the quadratic. It is expressed in quantities the iteration already holds, $n_*^\top z$ being needed for t_2 anyway. And it handles the reversed step of Remark 4.3 without a special case: there $t < 0$ and u_* accumulates negatively, so $t/2 + u_* < 0$ and the product with t is again positive. The value therefore increases on every nonzero step, in both directions, which is the monotonicity Section 5 needs.

5 Termination

Three questions are separate and are best kept so: does the inner loop stop, does the outer loop stop, and what does it mean when the iteration gets stuck.

5.1 The inner loop

Proposition 5.1 (The inner loop terminates)**.** *For a fixed entering constraint with $n_* \neq 0$, the inner loop performs at most $k + 1$ passes, where k is the size of the working set on entry.*

Proof. Every pass that does not exit performs a partial step and removes one constraint from the working set, so after at most k of them the working set is empty. With $\mathcal{A} = \emptyset$ we have $J_2 = J$ and hence, by Lemma 4.1(ii) and (10), $n_*^\top z = z^\top G z$ with $z = JJ^\top n_* = G^{-1} n_*$, so $n_*^\top z = n_*^\top G^{-1} n_* > 0$ because $G^{-1} \succ 0$ and $n_* \neq 0$. Thus $z \neq 0$, t_2 is finite by (24), and $t_1 = +\infty$ because the ratio test (23) ranges over an empty set. The step is full and the loop exits. □

The excluded case $n_* = 0$ is a column of C that is identically zero. Such a constraint reads $0 \geq b_*$: no x influences it, so it is either vacuous ($b_* \leq 0$) or renders the problem infeasible outright. The implementation scores it as infinitely violated when $b_* > 0$, which routes it to the infeasibility verdict below instead of to a division by zero in (19).

Algorithm 1 The Goldfarb–Idnani dual active-set method

Require: $G \succ 0$, a, C, b, m_{eq}

$J \leftarrow \mathrm{chol}(G)^{-1}$; $x \leftarrow G^{-1}a$; $f \leftarrow -\frac{1}{2}a^\top x$; $\mathcal{A} \leftarrow \emptyset$; $k \leftarrow 0$; $u \leftarrow ()$
loop
  $s \leftarrow C^\top x - b$, with $s_i \leftarrow 0$ for $i \in \mathcal{A}$; choose i_* by (19)
  if none **then return** (x, f, u) ▷ optimal, by Prop. 2.1
  $n_* \leftarrow c_{i_*}$; $u_* \leftarrow 0$
  loop
    $d \leftarrow J^\top n_*$; $z \leftarrow J_2 d_2$; $r \leftarrow R^{-1}d_1$
    $t_1, i_{\mathrm{del}} \leftarrow$ (23); $t_2 \leftarrow$ (24); **if** both infinite **then stop**: infeasible
    $t \leftarrow \min(t_1, t_2)$; $x \leftarrow x + tz$; $f \leftarrow f + t(t/2 + u_*)\, n_*^\top z$; $u \leftarrow u - tr$; $u_* \leftarrow u_* + t$
    if $t_2 \le t_1$ **then**
      $\mathcal{A} \leftarrow \mathcal{A} \cup \{i_*\}$, $u_{k+1} \leftarrow u_*$, $k \leftarrow k + 1$; insert (§6.1); **break**
    else
      $\mathcal{A} \leftarrow \mathcal{A} \setminus \{i_{\mathrm{del}}\}$, $k \leftarrow k - 1$; delete (§6.2); $s_{i_*} \leftarrow n_*^\top x - b_*$
    end if
  end loop
end loop

5.2 The outer loop

Proposition 5.2 (Strict ascent and finiteness)**.** *Every outer iteration ending in a full step with $t_2 \neq 0$ strictly increases the objective, hence by Proposition 2.3 the dual value ψ. If every full step has $t_2 > 0$, the method terminates after finitely many outer iterations.*

Proof. The iteration's total increment is the sum of (25) over its inner passes, each positive by Proposition 4.3, since the accumulated u_* and each step t share a sign throughout by Remark 4.3, and the final term is nonzero because $t_2 \neq 0$. For the second claim, a working set determines (x, u) uniquely by Proposition 2.2, hence determines the objective; that value strictly increases from one outer iteration to the next, so no working set recurs. There are finitely many subsets of $\{1, \dots, m\}$, and the iteration stops only by the optimality test of (19) or by the infeasibility verdict. □

Remark 5.1 (Degeneracy)**.** *The hypothesis fails exactly when a full step of length zero occurs, which requires $s_{i_*} = 0$: more constraints pass through x than the working set holds. A zero step then adds one without changing x, u or the objective, the ascent argument goes silent, and a cycle is not excluded. Goldfarb and Idnani [8] discuss this; the classical remedies are lexicographic or least-index rules. In finite precision the question changes character, since a slack of exactly zero is not what one observes.*

5.3 Infeasibility is a certificate, not a failure

The interesting case is a stuck iteration: $z = 0$, so the primal cannot move, and $t_1 = \infty$, so no multiplier limits the step. The classical reading is that the dual is unbounded, and therefore the primal infeasible. That argument needs care: by Proposition 4.2 the whole ray is dual feasible, and by (25) with $z = 0$ the value is constant. The cleanest statement avoids the dual entirely: the vector r that the solver already holds is a Farkas certificate.

Proposition 5.3 (Farkas certificate)**.** *Suppose the iteration reaches a state in which the entering inequality constraint i_* is violated, $s_{i_*} < 0$, and*

(a) $z = 0$, *and*

(b) $r_i \le 0$ *for every inequality* $i \in \mathcal{A}$.

Then the feasible set Ω *is empty.*

Proof. By (a) and Lemma 4.1(iv), $n_* = Ar = \sum_{i\in\mathcal{A}} r_i c_i$. Let $\tilde{x} \in \Omega$ be any feasible point. Splitting the working set into its equality part $\mathcal{E}$ and inequality part $\mathcal{I}$,

$$n_*^\top \tilde{x} = \sum_{i\in\mathcal{E}} r_i\, c_i^\top \tilde{x} + \sum_{i\in\mathcal{I}} r_i\, c_i^\top \tilde{x} \;\le\; \sum_{i\in\mathcal{E}} r_i b_i + \sum_{i\in\mathcal{I}} r_i b_i \;=\; r^\top b_{\mathcal{A}},$$

where the equality members contribute $c_i^\top \tilde{x} = b_i$ exactly, and each inequality member contributes $r_i c_i^\top \tilde{x} \le r_i b_i$ because $c_i^\top \tilde{x} \ge b_i$ and $r_i \le 0$ by (b). On the other hand the current iterate x satisfies $A^\top x = b_{\mathcal{A}}$, so

$$r^\top b_{\mathcal{A}} = r^\top A^\top x = n_*^\top x = s_{i_*} + b_* < b_*.$$

Combining, $n_*^\top \tilde{x} < b_*$, contradicting $\tilde{x} \in \Omega$. Hence $\Omega = \emptyset$. □

The proposition says the solver's infeasibility verdict is a proof rather than an exhausted search, and it says what the proof is: the multipliers r of the entering normal in terms of the working-set normals, which the solver computed for its own purposes, are the Farkas multipliers. The reversed case of Remark 4.3, an equality violated from above, follows by applying the proposition to the constraint $-n_*^\top x \ge -b_*$, which the equality implies.

Hypothesis (a) is $t_2 = \infty$ and (b) is $t_1 = \infty$, so the condition an implementation tests, neither step limit being finite, is literally the hypothesis of Proposition 5.3.

Remark 5.2 (The remaining hypothesis is not free in floating point)**.** *Proposition 5.3 also needs the entering constraint to be genuinely violated. Exact arithmetic gives that: a constraint is selected only when* $v_{i_*} > 0$. *Floating point does not, and the gap is reachable by the very substitution Section 6.1 argues for. A Householder reflection and a Givens chain leave the iterate at slightly different points, so a constraint one implementation leaves inside its boundary by a few multiples of the machine epsilon, another can leave the same distance outside. Reaching the stuck state with such a constraint selected, a solver must neither enforce it, there being nothing to enforce, nor conclude infeasibility, the conclusion not following; the implementation studied here sets it aside for the current iterate and takes the next candidate, discarding that judgement as soon as the iterate moves. The test separating rounding from a real violation is comparatively easy to set, because it need only separate rounding from* provable *infeasibility, and infeasibility is macroscopic: its size is set by the geometry of the constraints rather than by the arithmetic, so the two populations lie orders of magnitude apart.*

6 Updating the factorisation

Each iteration changes the working set by one column, and the factors must follow. Recomputing them from scratch costs a QR factorisation of $J^\top A$, $O(nk^2)$; the updates below cost $O(n^2)$ and $O(nk)$ respectively. Over the $O(n)$ iterations of a run that is the difference between $O(n^3)$ and $O(n^4)$, and the whole reason the factors are carried and not rebuilt.

Both updates work the same way. The invariant (10) is preserved by post-multiplying J with any orthogonal W, since $(JW)^\top G(JW) = W^\top W = I$; the update's task is to choose W so that (11) is restored for the new working set. Because $J' = JW$ gives $J'^\top A = W^\top (J^\top A)$, the same $W^\top$ acts on the rows of R.

6.1 Insertion

Let the working set of size k be extended by the entering constraint, so $A' = [\,A \;\; n_*\,]$. From (11) and $d = J^\top n_*$,

$$J^\top A' = \begin{bmatrix} R & d_1 \\ 0 & d_2 \end{bmatrix}, \tag{26}$$

which fails to be triangular only in the $n-k$ entries of d_2 below its first. Choose an orthogonal $W_2 \in \mathbb{R}^{(n-k)\times(n-k)}$ with $W_2^\top d_2 = \alpha e_1$ and set $W = \operatorname{diag}(I_k, W_2)$. Then

$$(JW)^\top A' = \begin{bmatrix} R & d_1 \\ 0 & W_2^\top d_2 \end{bmatrix} = \begin{bmatrix} R' \\ 0 \end{bmatrix}, \qquad R' = \begin{bmatrix} R & d_1 \\ 0 & \alpha \end{bmatrix}, \tag{27}$$

upper triangular of order $k+1$, and the leading k columns of J are untouched because W acts as the identity on them.

Proposition 6.1 (Full column rank is maintained)**.** *The insertion is performed only after a full step, which requires $t_2 < \infty$ and hence $z \neq 0$. Then $\alpha \neq 0$, so R' is nonsingular and A' has full column rank $k+1$.*

Proof. $|\alpha| = \|W_2^\top d_2\| = \|d_2\|$, which is nonzero iff $z \neq 0$ by Lemma 4.1(ii). R' is triangular with diagonal that of R extended by α, so it is nonsingular, and $R'^\top R' = A'^\top G^{-1} A'$ is then positive definite, which forces A' to have full column rank. □

The algorithm therefore never has to test for linear dependence among the working-set normals. The step rules exclude it. A constraint whose normal lies in the span of those already held has $z = 0$ by Lemma 4.1(iv), so its step is limited by t_1 and it can only be added after enough partial steps have removed the dependence. This is the structural advantage that Section 7 gives up.

The choice of W_2. Any orthogonal W_2 with $W_2^\top d_2 \in \mathbb{R}e_1$ will do, and by Proposition 3.2 the choice does not affect the iterates. The reference implementation uses a chain of $n-k-1$ Givens rotations, one per trailing component. A single Householder reflection achieves the same reduction:

$$W_2 = I - \frac{2}{\beta} v v^\top, \qquad v = d_2 - \alpha e_1, \qquad \beta = v^\top v, \qquad \alpha = \pm\|d_2\|. \tag{28}$$

That $W_2 d_2 = \alpha e_1$ is the standard Householder identity, using $\beta = 2\, v^\top d_2$. The reflection is symmetric, so it applies to the columns of J and to d_2 as one operation, and applying it to a block is a matrix–vector product followed by a rank-one update, two operations of $O(n(n-k))$, where the Givens chain is $n-k-1$ operations of $O(n)$. The arithmetic is comparable; the number of operations is not.

Remark 6.1 (The sign of α, and cancellation)**.** *Numerical analysis prescribes $\alpha = -\operatorname{sign}((d_2)_1)\|d_2\|$, which makes $v_1 = (d_2)_1 - \alpha$ a sum of like-signed terms. The implementation takes the opposite sign, $\alpha = \operatorname{sign}((d_2)_1)\|d_2\|$, so as to reproduce the sign convention of the reference's Givens chain, and thereby walks into the cancellation the prescription exists to avoid, severely when d_2 is already close to a positive multiple of e_1. The cure is algebraic rather than a change of convention: with $\tau = \|(d_2)_{2:}\|^2$ and $\nu = \|d_2\| = \sqrt{(d_2)_1^2 + \tau}$,*

$$v_1 = (d_2)_1 - \operatorname{sign}((d_2)_1)\,\nu = \frac{(d_2)_1^2 - \nu^2}{(d_2)_1 + \operatorname{sign}((d_2)_1)\nu} = \frac{-\operatorname{sign}((d_2)_1)\,\tau}{|(d_2)_1| + \nu}, \tag{29}$$

in which every operation combines like-signed quantities. The remaining entries of v are those of d_2, and $\beta = \tau + v_1^2$. By Proposition 3.2 the sign convention is anyway immaterial to the iterates; (29) is what makes it immaterial to the accuracy as well. See [9, 10] for the standard treatment.

Remark 6.2 (Why the reflections cannot be blocked)**.** *A sequence of Householder reflections is normally applied as a block. The WY representation of Bischof and Van Loan [3], and its storage-efficient form due to Schreiber and Van Loan [18], write a product $W_1 W_2 \cdots W_j$ as $I + WY^\top$, so that j reflections reach a matrix in one level-3 update rather than j level-2 ones. Blocked QR factorisation is fast for that reason, and the first thing to reach for here.*

It is not available, for a structural reason rather than an implementation one. Blocking requires the j reflections to be known before any is applied. In this iteration the reflection that inserts a constraint is determined by $d = J^\top n_*$, which depends on J after *the previous insertion; and between any two insertions the solver reads z and r off the updated factors to decide which constraint enters next, and whether it enters at all. The updates are interleaved with reads of what they produce, so there is no window in which two reflections are simultaneously known and unapplied.*

This is the same obstruction as in the deletion chase below, and the reason the method stays a level-2 computation no matter how it is coded. A variant that inserted several constraints per iteration would open the window; the dual method's step rules, which admit one constraint at a time by construction, close it.

6.2 Deletion

Let the constraint in position p of the working set be removed. Deleting column p of A deletes column p of R and shifts columns $p+1, \dots, k$ one place left, each retaining its own entries. The result is upper triangular except for one subdiagonal: writing $\tilde{R}$ for the shifted $k \times (k-1)$ matrix, $\tilde{R}_{ij} = 0$ for $i > j+1$, with the offending entries $\tilde{R}_{j+1,j}$ for $j = p, \dots, k-1$.

These entries lie in distinct columns, and no single reflection annihilates components of distinct vectors. What restores the shape is a *chase* of $k - p$ plane transformations: for $j = p, \dots, k-1$ in order, a 2×2 orthogonal acting on rows j and $j+1$ annihilates $\tilde{R}_{j+1,j}$, and the same transformation is applied to columns j and $j+1$ of J. The chase is inherently sequential, the parameters of each transformation being read from entries the previous one wrote, which is why no batched form of it exists.

The implementation takes the 2×2 block to be the symmetric reflection

$$W_j^{(2)} = \begin{bmatrix} c & s \\ s & -c \end{bmatrix}, \qquad c = \frac{x}{h}, \quad s = \frac{y}{h}, \qquad h = \operatorname{sign}(x)\sqrt{x^2 + y^2}, \tag{30}$$

where $x = \tilde{R}_{jj}$ and $y = \tilde{R}_{j+1,j}$, so that $W_j^{(2)}(x, y)^\top = (h, 0)^\top$. Symmetry is the point. The update rule of this section applies W to the columns of J and $W^\top$ to the rows of R; a symmetric block makes those the same matrix, so one 2×2 serves both sides. A Givens rotation, which is what BLAS offers, is not symmetric: it agrees with (30) on the first output and negates the second, so it requires its transpose on the R side. By Proposition 3.2 either choice gives the same iterates, the difference being a sign matrix.

Remark 6.3 (The degenerate step)**.** *When $x = 0$ the formula (30) gives $c = 0$, $s = \pm 1$, and the transformation reduces to an exchange of the two rows up to sign. The implementation performs a plain exchange, $\left[\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right]$, which is orthogonal and symmetric and annihilates y just as well; it differs from (30) by a sign when $y < 0$, and Proposition 3.2 says that is free.*

Remark 6.4 (Where the storage cost lands)**.** *R is held as packed columns, entry (i, j) with $i \le j$ at offset $j(j+1)/2 + i$, so its leading $k \times k$ block, which the solve for r reads once per iteration, is a contiguous run at every k. Insertion writes one column, contiguous in that layout; deletion mixes two* rows *across a range of columns, which is a strided gather because column offsets grow with the index. The asymmetry is deliberate, and the right way round. It is a property of the layout rather than of the derivation, to which the mathematics is indifferent.*

7 Guessing the active set instead of walking to it

The dual method reaches the active set one constraint at a time. That is its defining feature and, at moderate n, its principal cost: a budget-plus-bounds problem in $n = 100$ variables takes some 13 outer iterations (Section 9.1), each of which is a handful of matrix–vector products. The alternative is to guess the whole active set at once, solve the working-set subproblem it induces, and repair the guess from the signs that come back. This is the semismooth Newton method of Hintermüller, Ito and Kunisch [11] read as a primal–dual active-set strategy, and block principal pivoting on the associated linear complementarity problem [12, 14, 5] read as a rule for exchanging several indices at a time.

7.1 The iteration

Given a candidate set $\mathcal{A}$, Proposition 2.2 gives the working-set solution directly:

$$(C_{\mathcal{A}}^{\top} G^{-1} C_{\mathcal{A}})\lambda_{\mathcal{A}} = b_{\mathcal{A}} - C_{\mathcal{A}}^{\top} x_u, \qquad x = x_u + G^{-1} C_{\mathcal{A}} \lambda_{\mathcal{A}}, \qquad x_u = G^{-1} a, \tag{31}$$

with $\lambda_i = 0$ for $i \notin \mathcal{A}$. Both solves go through one Cholesky factorisation of G, computed once, and one of the $k \times k$ dual Hessian, computed per repair. The repair rule reads the two sign conditions of (5)–(4) as instructions:

$$\mathcal{A}' = \{1, \dots, m_{\text{eq}}\} \cup \{\, i > m_{\text{eq}} \;:\; (i \in \mathcal{A} \text{ and } \lambda_i \ge 0) \;\text{ or }\; (c_i^{\top} x < b_i) \,\}, \tag{32}$$

that is: an active constraint whose multiplier has gone negative does not belong in the set and leaves it; an inactive constraint that is violated at x does belong and joins it; equalities are always in. The starting guess is the equalities together with whatever the unconstrained minimiser violates, which is already the answer when it violates nothing. If $\mathcal{A}' = \mathcal{A}$ the conditions (4)–(6) hold to the tolerance used, and (31) supplies (3) by construction.

The whole set is exchanged at once, and it converges in a handful of repairs almost independently of n, from 1.3 to 3.5 across every family and size measured in Section 9.1. It trades arithmetic, which is abundant at these sizes, for iterations, which are not.

7.2 Why there is no finite-termination theorem here

Block exchange can cycle, and the standard remedy is a least-index rule that flips only the lowest-indexed offending constraint, in the manner of Bland's rule for the simplex method and Murty's least-index rule for linear complementarity problems [14]. What follows concerns this *block* exchange specifically, and not active-set methods in general: the exact walk of Section 4 is finite under nondegeneracy (Proposition 5.2), and for parametric families of QPs its worst-case iteration count can be certified exactly [1]. In the bound-constrained case that remedy comes with a theorem. The theorem does not survive the generalisation, and the reason is structural, not a gap in the argument.

Consider $\min_{x\geq 0} \frac{1}{2}x^\top Gx - a^\top x$, so that $C = I$ and $m = n$. The system solved on a candidate set $\mathcal{A}$ is then

$$(I_{\cdot\mathcal{A}}^\top G^{-1} I_{\cdot\mathcal{A}})\lambda_\mathcal{A} = (G^{-1})_{\mathcal{A}\mathcal{A}}\lambda_\mathcal{A} = b_\mathcal{A} - x_{u,\mathcal{A}}, \tag{33}$$

whose coefficient matrix is a *principal submatrix* of G^{-1}. Since $G^{-1} \succ 0$, every principal submatrix of it is positive definite, so G^{-1} is a P-matrix: every principal pivot is well defined, whatever the guess. That is the hypothesis under which block principal pivoting with a least-index guard terminates finitely at the unique solution [12, 14], and the guarantee survives inexact inner solves once their residual is small against the problem's decision margin [17].

None of it is available for general C. The matrix $C_\mathcal{A}^\top G^{-1} C_\mathcal{A}$ is positive definite exactly when $C_\mathcal{A}$ has full column rank, and that is a property of the guess, not of the data: a guessed set may name $k > n$ constraints, or $k \leq n$ linearly dependent ones, and then the pivot is undefined. There is no P-matrix to appeal to, and correspondingly no theorem to inherit. Contrast Proposition 6.1: the dual method's step rules make dependence unreachable, and guessing forfeits that protection.

The implementation treats the Cholesky factorisation of $C_\mathcal{A}^\top G^{-1} C_\mathcal{A}$ as the rank test, where a dependent guess fails instead of returning something plausible, and the least-index rule as a way of making progress where block exchange oscillates, and not as a termination proof. Neither substitutes for the guarantee. The certificate does.

7.3 The certificate

Because the method may fail, and may fail by stabilising on a set that is simply not optimal, the trade is sound only because the answer is checked. Here Proposition 2.1 carries the argument: a candidate (x, λ) satisfying

$$\|Gx - a - C\lambda\|_\infty \leq \varepsilon, \quad |c_i^\top x - b_i| \leq \varepsilon \ (i \leq m_{\text{eq}}), \quad c_i^\top x - b_i \geq -\varepsilon, \ \ \lambda_i \geq -\varepsilon, \ \ |\lambda_i(c_i^\top x - b_i)| \leq \varepsilon \tag{34}$$

on the inequalities, with ε scaled by the data, *is* the answer to that tolerance. Stationarity is checked against G directly and not trusted from the construction, since the construction is what an ill-conditioned working set corrupts. A candidate that fails is discarded and the exact walk of Algorithm 1 runs instead, so guessing can never degrade the answer: it produces the minimiser or nothing.

Remark 7.1 (Two tolerances with very different jobs)**.** *The tolerance in* (32) *decides which set is tried next. A bad choice there costs a repair or a fallback and can never cost correctness, so it is not delicate. The tolerance in* (34) *is the only thing standing between a non-optimal point and the caller, and is therefore the one to reason about. It is nonetheless not delicate either, because the quantity it thresholds is bimodal: a candidate constructed from a well-conditioned working set satisfies* (34) *to a few multiples of the machine epsilon, and one built on a set that is not has no reason to come close. Section 9.1 measures the accepted population, whose worst residual is* 2.6×10^{-13}*, and reports what it did and did not observe on the rejecting side.*

8 A family of programs differing only in the linear term

An efficient frontier, a rolling rebalance and a scenario grid all solve the same program repeatedly with a slightly different linear term: G, C, b and m_{eq} are fixed and only a varies. Solved independently, each instance rediscovers an active set it almost always already had.

What makes this exploitable is a reading of the invariants (10)–(11): *neither factor depends on a.* J depends on G alone through $J^\top GJ = I$ and on the working set through the orthogonal

transformations accumulated into it; R depends on G and the working set. The linear term enters only through $x_u = G^{-1}a$ and the right-hand sides. So across such a family both factors are reusable verbatim, and the entire solution can be recovered from them.

8.1 Recovery

Proposition 8.1 (Recovery from cached factors)**.** *Let (J, R) satisfy (10)–(11) for a working set $\mathcal{A}$ of size k with normals A. For a new linear term a, set*

$$x_u = J\,(J^\top a), \qquad \rho = b_{\mathcal{A}} - A^\top x_u, \qquad R^\top y = \rho, \qquad x = x_u + J_1 y, \qquad R\,u = y. \tag{35}$$

Then (x, u) is the solution and multiplier pair of the working-set subproblem (7).

Proof. $x_u = JJ^\top a = G^{-1}a$ by (10). By Proposition 3.1(i), $u = R^{-1}y = R^{-1}R^{-\top}\rho = (R^\top R)^{-1}\rho = (A^\top G^{-1}A)^{-1}\rho$, which is the multiplier of (8). For the iterate, $Ru = y$ gives $J_1 y = J_1 R u = G^{-1}Au$ by Proposition 3.1(iii), so $x = x_u + G^{-1}Au$, again as in (8). □

The cost is $O(n^2)$, all of it in $x_u = J(J^\top a)$: two matrix–vector products with a dense $n \times n$ matrix, J having lost its triangularity at the first update. Everything after that is $O(nk + k^2)$. Re-deriving the factors for the same working set instead costs k insertions at $O(n^2)$ each, so the saving is a factor of order k, largest exactly where it matters: a long-only optimum is a vertex at which most bounds bind.

The exponent is easy to get wrong, because the natural experiment cannot see it. On a box family the active set grows with n, so $O(nk)$ and $O(n^2)$ predict the same curve and either reading fits; holding k fixed at 5 while n grows separates them, and the measured local exponent $d\log t/d\log n$ then runs from 0.13 at $n = 100$ to 2.96 at $n = 1600$. Neither is 1. The flat small-n end is an overhead floor, a hit there costing what its dozen array operations cost to dispatch, and it ends where the n^2 overtakes dispatch.

Whether (x, u) from (35) is the answer to the *full* program is then one certificate away, the same certificate as before: the multipliers of the working-set inequalities must be non-negative and no constraint outside the working set may be violated. By Proposition 2.1 that is a proof. Note the asymmetry in cost: the recovery is $O(n^2)$ but the check must look at every constraint, so it is $O(nm)$ for a dense C, so verification and not recovery is what dominates a successful reuse on a dense problem.

8.2 Repair rather than restart

When the check fails, the cached working set is stale, but it remains a far better starting point than the unconstrained minimum. What blocks resuming from it is (18): the iteration requires dual feasibility, and a stale set typically has one or more negative multipliers. Those are precisely the constraints that no longer belong.

Proposition 8.2 (Repair terminates in a resumable state)**.** *Iterate the following from the cached set: recover (x, u) by (35); if every inequality multiplier is non-negative, stop; otherwise drop one constraint with a negative multiplier, downdate (J, R) by Section 6.2, and repeat. The loop stops after at most k passes, in a state satisfying (18).*

Proof. Each pass that does not stop shrinks the working set by one, so there are at most k; with an empty set the sign condition is vacuous, which is the cold start of Remark 2.1. On exit (x, u) solves the subproblem by Proposition 8.1 and the sign conditions hold, which is (18); full column rank is inherited, deleting columns preserving it. □

From such a state the iteration of Section 4 cannot tell a resumed solve from a cold one, the invariant being all it reads, so the resumed walk needs only to drive the remaining primal infeasibility to zero. In the worst case everything is dropped and the resumed walk is a cold solve, so the repair is never worse than restarting up to the cost of the drops themselves.

Remark 8.1 (Why this cannot change the answer)**.** *Neither the reuse nor the repair introduces an approximation. The reuse returns a point only when it carries a certificate, and the repair merely chooses a different starting state for an iteration whose output is determined by Proposition 2.1. What does change is the reported iteration count, which counts the working-set edits actually performed and is therefore not comparable with a cold solve's.*

9 Numerical experiments

Everything above is a proof, and proofs leave two things open. The first is whether the code satisfies the identities: they hold in exact arithmetic, but the implementation reaches them through a triangular factor held as packed columns addressed by hand and through library calls writing in place into a view of J's own buffer, correct only while that view stays contiguous. A sign error in the packed indexing would leave every proposition above true and the running solver wrong, and wrong quietly: the invariants are maintained across a run, not recomputed, and a solver drifting from $J^\top G J = I$ still returns a vector that is very nearly right. The second is the set of quantities the paper reasons about but cannot predict: how often a guessed active set is certifiable, how many repairs it takes, what a warm-started solve costs. All measurements are on the implementation at the version recorded in Section 11, by scripts distributed with this paper's sources.

9.1 The guessed active set, and what the certificate catches

Section 7 makes two claims only measurement settles: that the block exchange converges in a number of repairs independent of n, and that losing the P-matrix property for general C is a real exposure. Figure 1 addresses both over 5 constraint families, 30 instances each, at $n \in \{12, 25, 50, 100, 200\}$, instrumenting the shipped routines instead of reimplementing them.

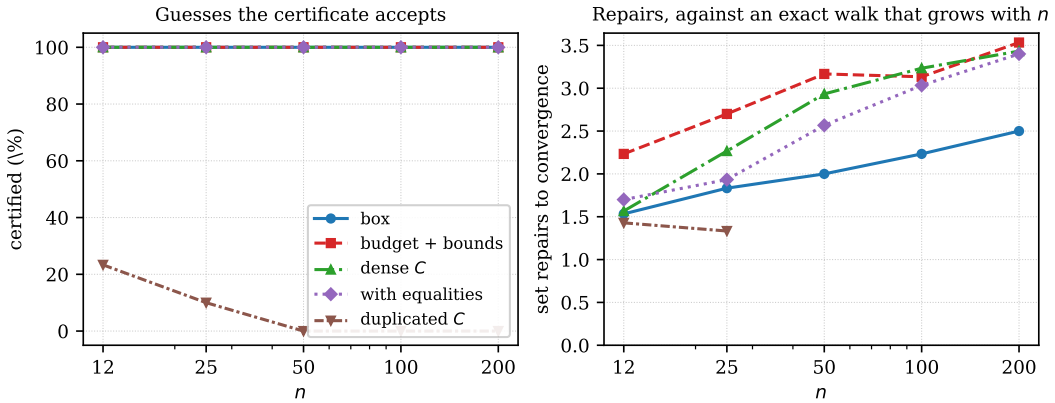

Figure 1: Left: the fraction of guesses the certificate accepts. Right: set repairs to convergence. Four families are certified without exception at every size and settle in 1.3 to 3.5 repairs while the exact walk's outer iterations grow linearly in n. The fifth is constructed so that a guessed set can be linearly dependent, and it fails.

On the four well-posed families (bounds, budget-plus-bounds, dense C, and C with equalities) every guess is certified at every size, and the repair count rises only from 1.3 to 3.5 across a sixteen-fold range of n, never exceeding 5. The comparison that matters is against the exact walk on the same instances: on budget-plus-bounds it takes 13 outer iterations at $n = 100$, and its count grows linearly where the repair count is very nearly flat. Section 7 describes that trade; here it is measured.

The rank-deficient guess is real, and invisible by accident. Over 30 instances at each of five sizes, not one guess on those four families produced a rank-deficient working-set system, from which a reader might conclude the exposure is theoretical. It is not, merely invisible in random data, where a dense C almost never has a dependent subset the guess happens to name. The fifth family makes it reachable in the most elementary way, by repeating columns of C, so that naming both copies induces a singular system: there the rank test fires on 93.3% of attempts and the certified fraction collapses from 23% at $n = 12$ to 0% at $n = 200$, against 0.0% on the others. Duplicated constraints are not exotic, arising whenever a model is assembled programmatically from overlapping rules, and they give the structural point of Section 7 its concrete form: for bounds the system is a principal submatrix of G^{-1} and cannot be singular, and nothing about general C preserves that.

Where the failures are caught. Of the candidates reaching the certificate, 610 were accepted with a worst KKT residual (34) of 2.6×10^{-13}, and 0 were rejected. We report that asymmetry instead of smoothing it: on these families the certificate never had to reject a converged candidate, because the failures were caught earlier and more cheaply by the Cholesky factorisation refusing a dependent guess. The certificate remains the guarantee, but on this evidence the rank test does the work in practice.

9.2 Against other solvers, and against the same method done differently

The experiments so far compare the solver with its own claims, which establishes that it is implemented correctly and says nothing about whether the method is worth using. Table 1 and Figure 2 answer that against four alternatives. Two are outside the family: an operator-splitting method (OSQP) and an interior-point method (Clarabel). Two are inside it, and are the informative ones: the reference C implementation of this exact algorithm, and DAQP [2], which walks the same dual active set but carries it on recursive $LDL^\top$ updates of the working-set system rather than on the (J, R) pair of Section 3.

Times alone would mislead, in the direction that flatters whichever solver was configured loosest: an active-set method returns an exact KKT point, a splitting method stops at a tolerance, and an interior-point method approaches the boundary without reaching it. Every solver is therefore asked for 10^{-9} and the residual it *achieves* is reported beside its time, in the scaled sup-norm of (34). Multipliers are recovered from the returned iterate by least squares and not read from the solver, so none is charged for reporting them in another convention.

The same algorithm in two languages crosses over. The C reference is the fastest solver here at $n = 50$ and among the slowest at $n = 400$, where it is 5.5× slower than the implementation measured in this paper. Nothing algorithmic separates them: they walk the same active sets in the same order. What separates them is that below a few hundred variables a solve costs what its operations cost to dispatch, and above it a solve costs arithmetic, which reaches vendor-tuned kernels in one and hand-written scalar loops in the other. This reproduces, independently and as a by-product, the crossover the implementation is designed around.

Table 1: Time per solve in milliseconds and achieved KKT residual, at $n \in \{50, 100, 200, 400\}$. The first four rows are active-set methods and the last two are not, which the residual column is there to show.

	$n = 50$		$n = 100$		$n = 200$		$n = 400$	
Solver	ms	resid.	ms	resid.	ms	resid.	ms	resid.
box								
cvx-quadprog	0.07	6×10^{-16}	0.11	1×10^{-15}	0.30	2×10^{-15}	1.02	3×10^{-15}
cvx-quadprog, `fast`	0.07	6×10^{-16}	0.08	1×10^{-15}	0.13	2×10^{-15}	0.33	3×10^{-15}
`quadprog` (C)	0.02	1×10^{-15}	0.10	6×10^{-16}	0.60	1×10^{-15}	5.64	1×10^{-15}
DAQP	0.03	8×10^{-16}	0.11	6×10^{-16}	0.88	2×10^{-15}	8.59	2×10^{-15}
OSQP	0.47	1×10^{-15}	1.00	2×10^{-15}	3.54	2×10^{-15}	15.20	3×10^{-15}
Clarabel	0.57	6×10^{-16}	2.41	7×10^{-16}	8.43	1×10^{-15}	34.43	1×10^{-15}
budget + bounds								
cvx-quadprog	0.15	1×10^{-15}	0.32	2×10^{-15}	0.63	3×10^{-15}	3.75	1×10^{-14}
cvx-quadprog, `fast`	0.15	1×10^{-15}	0.22	2×10^{-15}	0.53	4×10^{-15}	1.13	9×10^{-15}
`quadprog` (C)	0.03	1×10^{-15}	0.18	1×10^{-15}	1.16	3×10^{-15}	12.92	6×10^{-15}
DAQP	0.04	1×10^{-15}	0.19	2×10^{-15}	1.57	3×10^{-15}	13.34	1×10^{-14}
OSQP	0.56	1×10^{-15}	1.47	2×10^{-15}	5.68	3×10^{-15}	25.83	1×10^{-14}
Clarabel	0.66	5×10^{-8}	3.08	4×10^{-8}	10.20	2×10^{-6}	38.39	2×10^{-7}
dense C								
cvx-quadprog	0.24	5×10^{-15}	0.52	7×10^{-15}	1.60	1×10^{-14}	7.01	2×10^{-14}
cvx-quadprog, `fast`	0.12	4×10^{-15}	0.20	9×10^{-15}	0.44	1×10^{-14}	1.18	2×10^{-14}
`quadprog` (C)	0.04	4×10^{-15}	0.23	7×10^{-15}	2.03	1×10^{-14}	19.17	2×10^{-14}
DAQP	0.03	6×10^{-15}	0.17	7×10^{-15}	1.31	1×10^{-14}	11.93	2×10^{-14}
OSQP	0.55	5×10^{-15}	1.90	8×10^{-15}	11.45	1×10^{-14}	67.05	2×10^{-14}
Clarabel	1.76	2×10^{-9}	5.61	7×10^{-10}	73.92	9×10^{-8}	106.95	4×10^{-5}

Accuracy is where the interior-point method pays. Clarabel is asked for 10^{-9} and delivers between 10^{-9} and 4×10^{-5}, against 2×10^{-14} for the active-set method, several orders of magnitude, and not a defect of the solver. The optima here are vertices of the feasible polyhedron, with many constraints active, and a vertex is exactly the case an interior-point method approaches asymptotically. OSQP reaches 2×10^{-14}, comparable to the active-set method, but only with its polishing step enabled and its tolerance set to 10^{-9}; at its default tolerance it is faster and not comparable. The general point is that on combinatorially structured optima an active-set method returns an answer the others can only approach, and comparing times without residuals hides that.

A different factorisation of the same walk crosses over too. Both in-family solvers behave the same way. Both are fastest of everything at $n = 50$, the C reference on bounds and on budget-plus-bounds and DAQP on dense C, and both are behind at $n = 400$, where DAQP is 8.43× slower on bounds and 1.70× on dense C. Accuracy is indistinguishable from ours throughout (2×10^{-14} at worst), as one expects of methods in the same family: all three terminate on a vertex rather than approaching one, which is exactly what separates them from the two rows below. The comparison is not apples to apples in one respect: DAQP is written in library-free C for embedded targets and pays no interpreter cost, so at small n it is measuring a different thing from us. What it shares is the shape of the crossover, and what it shows is that the shape is a property of the regime and not of any one implementation.

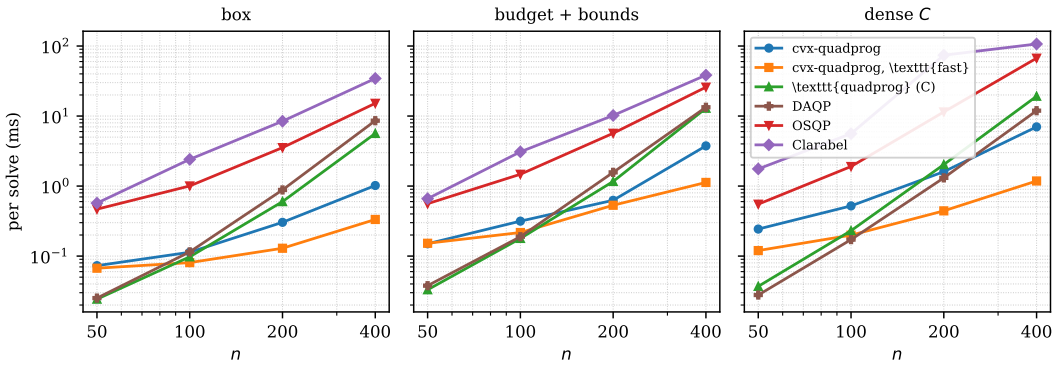

Figure 2: Time per solve against n on three constraint shapes. The reference implementation of the same algorithm wins at the small end, where cost is interpreter dispatch and not arithmetic, and loses at the large end, where it is arithmetic.

Guessing the active set is worth about what the language is. The `fast` path is the quickest of everything measured at $n = 400$ on every family, and it beats the exact walk by $3.1\times$ to $5.9\times$. That is the same order as the gap between the two implementations of the exact walk at the same size, $5.5\times$ on bounds and $2.7\times$ on dense C, and the two orderings run opposite: guessing gains most on dense C, where the language gains least, and least on bounds, where it gains most. Iterations, not the cost of an iteration, dominate at these sizes; Section 7's argument arrives here from a different direction.

What this comparison is not. These are dense problems of small to moderate size whose optima are vertices, which is the territory Section 1 claims for the method and precisely the territory OSQP and Clarabel are not built for. Both are designed for large sparse problems, both exploit sparsity this experiment does not give them, and both would be expected to win outright at n in the tens of thousands with a sparse G. The table is evidence that the method is the right choice on its own ground, and no evidence at all about anyone else's.

So which of the four should a reader use? The question is fair, and the table answers it only if one reads the columns against each other, so we state it. For dense problems whose optima are vertices, any of the four in-family rows returns the same answer to machine precision, and the choice is about constraints other than accuracy. Below a few hundred variables the compiled implementations win, and by a margin no amount of array-language tuning will close, because what is being measured there is interpreter dispatch. Above that the ordering reverses, and the fast path of Section 7 leads on every family measured. The two solvers outside the family are the right choice once n is large and G is sparse, which this experiment does not test and does not claim to.

DAQP deserves a sharper statement than the ordering gives it, because on three counts it is simply ahead: it handles semidefinite G through proximal regularisation, its worst-case iteration count can be certified offline for a parametric family [1], and it is library-free C sized for embedded targets. A reader with a toolchain, a real-time deadline and a semidefinite Hessian should use it. What the implementation studied here offers against that is narrower: no compiler and no build step, a drop-in signature for code already written against the reference, and the arithmetic advantage above a few hundred variables. Whether that is the better trade is a property of the reader's situation and not of the method.

9.3 What these experiments do not establish

These experiments verify agreement between the code and the identities proved here, not the absence of defects elsewhere, in constraint selection or the infeasibility verdict or degenerate input, for which a differential comparison of complete trajectories against an independent implementation would be the instrument. They say nothing about conditioning: problems use $G = BB^\top + nI$, and the residuals would grow with $\kappa(G)$. That choice also biases Section 9.1, in the direction that works against the conclusion drawn there, since a well-conditioned G spreads the minimiser so few bounds bind where a realistic covariance concentrates it and binds many; Section 10 measures how far off that is. And the checks are dense and of modest size, with single-machine timings.

10 A problem the synthetic families were not standing in for

Every experiment so far draws its Hessian as $G = BB^\top + nI$ for Gaussian B. Section 9.3 names that as a limitation and says the bias runs against the method the paper advocates. This section makes the claim concrete rather than leaving it as a caveat, by solving the problem the method was built for, the long-only portfolio, on two real return series.

The program is (2) with the budget as the single equality and non-negativity as the bounds,

$$\min_{w} \ \tfrac{1}{2} w^\top \Sigma w - \rho\, \mu^\top w \quad \text{subject to} \quad \mathbf{1}^\top w = 1, \quad w \geq 0, \tag{36}$$

with Σ the annualised sample covariance of daily returns and μ the sample mean. Two universes, so that nothing below is a property of one market: the S&P 500 at $n = 494$ assets over $T = 1213$ days, and the FTSE 100 at $n = 86$ over $T = 1698$.

The FTSE series carries one instrument quoted flat across the whole window. Its sample variance is zero, so Σ is singular, rank 86 of 87, and the solver refuses the problem, correctly: an asset with no variance has no place in a variance-minimising portfolio. The degenerate column is dropped and $n = 86$ is the result.

10.1 Real covariances are the hard case, and in the expected direction

Table 2 reports the minimum-variance corner, $\rho = 0$.

The differences from the synthetic families all run in the same direction.

Worse conditioning, and a far larger active set. $\kappa(\Sigma)$ is 1×10^5 on the S&P 500 against the $O(n)$ of a Gaussian $BB^\top + nI$: a sample covariance is close to low rank plus diagonal, hence ill-conditioned, and real data looks like that. At the minimum-variance corner the solver holds 442 of $494 + 1$ constraints active and the portfolio holds only 53 names. Long-only minimum variance concentrates onto a vertex at which almost every bound binds, where a well-conditioned Hessian spreads the optimum instead.

And so the exact walk is far longer. It takes 454 outer iterations here against the 13 that the synthetic budget-plus-bounds family produced at $n = 100$ in Section 9.1. The iteration count grows with the size of the set the walk must reach, and on real data that set is large. Every statement in Section 9.1 about the advantage of guessing the active set was therefore measured at the wrong end of its range: on the S&P 500 the fast path settles in 8 repairs against 454 outer iterations, and is 4.2× faster than the exact walk, against the 3.3× of the synthetic family with the same constraint shape at $n = 400$.

Table 2: Long-only minimum variance on real data: time per solve and achieved KKT residual. Sizes, conditioning and active-set size are given per universe.

Solver	ms	residual
S&P 500, $n = 494$, $T = 1213$, $\kappa(\Sigma) = 1 \times 10^5$, 442 active		
cvx-quadprog	38.6	2×10^{-15}
cvx-quadprog, `fast`	9.2	1×10^{-15}
`quadprog` (C)	127.2	3×10^{-15}
DAQP	59.4	1×10^{-15}
OSQP	64.4	9×10^{-16}
Clarabel	75.4	2×10^{-10}
FTSE 100, $n = 86$, $T = 1698$, $\kappa(\Sigma) = 5 \times 10^2$, 62 active		
cvx-quadprog	1.3	2×10^{-16}
cvx-quadprog, `fast`	0.4	3×10^{-16}
`quadprog` (C)	0.4	1×10^{-16}
DAQP	0.2	2×10^{-16}
OSQP	1.1	4×10^{-16}
Clarabel	2.2	5×10^{-5}

The crossover of Section 9.2 appears here too, with a point either side and on real instances constructed for neither purpose: on the S&P 500 the exact walk beats both in-family competitors, the C reference and DAQP [2], and on the FTSE 100, at $n = 86$, it loses to both.

Accuracy survives the conditioning. The active-set method returns 2×10^{-15} despite $\kappa(\Sigma) = 1 \times 10^5$, against five orders of magnitude more for the interior-point solver at the same requested tolerance, and eleven on the FTSE 100. Section 9.2's argument reaches its sharpest form here: the optimum is a vertex with 442 active constraints, which is the worst case for a method that approaches the boundary asymptotically and the natural case for one that terminates on it.

10.2 The frontier is what warm starting is for

A frontier sweep varies ρ in (36) and changes nothing else. That is precisely the family of Section 8: Σ, C and b fixed, only the linear term moving. Figure 3 traces 50 points across the S&P 500 frontier.

Reusing the factorisation costs 1.8 ms per point against 37.8 ms solving each point from scratch, a factor of 20.8, and the answers are identical to the cold solves because a reused point is returned only against the certificate of Section 8.

The interesting part is where that factor comes from, and it is not where one would expect. The cached active set stayed optimal on only 16 of the 50 points; the other 34 were misses. A sweep spanning three decades of ρ moves a long way between consecutive points, the right-hand panel showing the holdings falling from 55 names to 1 across it, and the active set moves with them. Sampling the frontier more finely would raise the hit rate directly; nothing here is tuned to produce one.

So the saving is not mostly the hit path. It is that a *miss* is repaired rather than restarted. Proposition 8.2 turns a stale set into a dual-feasible state by dropping the constraints whose multipliers have gone negative, and the iteration resumes from there instead of from the unconstrained minimum, keeping J and R throughout. A sweep that misses two points in three is still 20.8× faster than solving each point cold, which is a sharper statement about the machinery than a high hit rate would have been: it says the mechanism that matters is the one that runs when the cache is *wrong*.

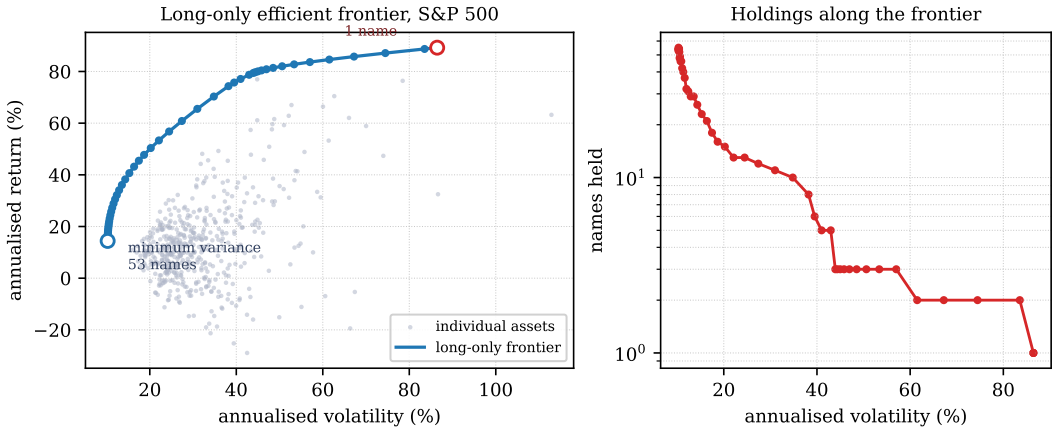

Figure 3: Left: the long-only efficient frontier, with the individual assets it dominates. Right: the number of names held along it, on a log scale, falling from 55 at the minimum-variance corner to 1 at the return-seeking end, where the budget concentrates in a single asset.

An earlier version of this experiment spaced ρ linearly. Almost every point then landed on the return-seeking corner, where the solution is a single asset and does not move, and the cache hit 48 times out of 50. That number was real and any conclusion drawn from it would have been worthless, because the sweep was solving the same problem forty-nine times instead of tracing a frontier. The geometric spacing is what makes the left-hand panel a curve, and the honest hit rate is the one that comes with it.

This section claims that the method's advantages are larger on the problems it was designed for than on the synthetic families used to test it, and that the direction of the gap was predicted, not found afterwards. It does not claim this is a good way to build a portfolio: minimum variance on a raw sample covariance is a poor estimator, and Σ is used here only as a source of realistically shaped matrices.

11 Summary

The Goldfarb–Idnani method is often presented as a sequence of updates, which makes it look more intricate than it is. Read from the invariants it is short. One matrix J is carried, defined by $J^\top G J = I$ and $J^\top A = [R; 0]$: its columns are a G-orthonormal basis of $\mathbb{R}^n$, arranged so that the trailing block spans the working set's feasible subspace. Every quantity the iteration needs is then one matrix–vector product or one triangular solve away, and every quantity it *consumes* has a closed form in (G, A, n_*) alone (Proposition 3.2), which is why the representation may be chosen for speed, and why two implementations holding different factors compute the same step.

The iteration is one affine path (Proposition 4.2) along which stationarity holds identically while the entering multiplier rises and the working-set multipliers fall. Two limits cut it short, and which binds decides whether a constraint is added or dropped. The objective advances by an exact closed form (Proposition 4.3), positive in both step directions, so the method is a dual ascent and, absent degeneracy, finite (Proposition 5.2). When the path is blocked in both directions the vector r is a Farkas certificate (Proposition 5.3): the infeasibility verdict is a proof, and the solver already holds it.

Two properties of the classical method are easy to overlook, and the extensions of Sections 7

and 8 trade one away and exploit the other. The first is that linear dependence among the working-set normals is unreachable: the step rules exclude it (Proposition 6.1), so no rank test is needed anywhere. Guessing the active set forfeits it, and a primal–dual active-set fast path therefore admits no finite-termination theorem for general constraints: there is no P-matrix structure to appeal to once the working-set system stops being a principal submatrix. The second is that the factors depend on G and the working set but not on the linear term, so a family of programs differing only in a is solvable from one factorisation, and a stale active set repairable rather than disposable.

Both extensions are unguaranteed and both are safe, for the same reason: strict convexity makes the KKT conditions sufficient (Proposition 2.1), so a candidate can be certified instead of trusted. The strict convexity that buys it is a choice of scope rather than a boundary of the approach: the dual method extends to semidefinite G, with finite termination, by Boland [4], and by proximal regularisation in the solver of [2]. Of the ideas here, that one travels furthest. A method that may fail, but that can recognise its own success, needs a certificate and not a convergence theory.

One observation generalises past this solver. Below a few hundred variables a solve costs what its array operations cost to dispatch, not what its arithmetic costs, so halving the flop count buys nothing and halving the *iteration* count buys everything: the fast path of Section 7 gains about as much over the exact walk as rewriting that walk in another language does, and gains most on the family where the rewrite gains least (Section 9.2). Effort spent on kernels is wasted where the bottleneck is the interpreter, and effort spent on iteration counts is wasted where it is not; knowing which regime one is in is the whole of the decision.

Reproducibility

Every number in this paper is produced by a committed script, and no figure or table was edited by hand. The scripts write the files the paper `\inputs`, so a stale figure cannot survive a regeneration unnoticed.

Script (`quadprog_note/`)	Figure	Table	Used in
`experiment_qp_identities`	—	—	below
`experiment_qp_pdas`	`quadprog_pdas`	`_pdas_defs`	§9.1
`experiment_qp_sweep`	—	`_sweep_defs`	§8
`experiment_qp_compare`	`quadprog_compare`	`quadprog_compare`	§9.2
`experiment_qp_portfolio`	`quadprog_portfolio`	`quadprog_portfolio`	§10

Each row's table entry stands for both the tabular fragment and the `_defs` file of macros beside it, where the script writes both. The identity script writes no figure, and the sweep script writes one the paper does not include: its cost curves are the natural way to inspect the two regimes §8 describes in words, so the figure is left in place for a reader who wants to see them.

The identities are checked against the code. A proof establishes the identities of Sections 3 to 8 in exact arithmetic; it does not establish that the implementation satisfies them, which matters because it reaches them through a packed triangular factor addressed by hand and through library calls writing in place into a view of J's own buffer. `experiment_qp_identities` evaluates both sides of all 28 of them against the solver's own internal state and not a reimplementation: 29696 checks over 560 factorisation states, worst relative residual 5.2×10^{-15}. Two come out exactly zero, and one of those is Remark 4.1.

Software under study. `cvx-quadprog`, version 0.4.2 [16], archived with a DOI. The measurements are tied to that version rather than to the latest release, so that the reference keeps pointing at the code that produced them. The experiments import the package's private modules where the claims are about private state, namely J, the packed R, and the vectors d, z, r, and they instrument the shipped routines by wrapping them instead of reimplementing them, so what is measured is what runs.

Regenerating everything. From the paper's source tree, `make figures` runs all five scripts and rewrites every generated file; `make compile` then rebuilds this document. Each script also runs standalone as a module. Setting `EXPERIMENT_SMOKE=1` shrinks every sweep so that the whole set completes in seconds, which is how the continuous-integration suite exercises each code path without reproducing the full runs, and `EXPERIMENT_OUT` redirects the outputs so that a reduced run cannot overwrite the committed ones.

Determinism. Every random problem is drawn from a seed derived arithmetically from the family, size and instance index, so the tables reproduce across runs and machines. Timings do not, and are not claimed to: they were taken on one machine against NumPy built on Apple Accelerate, with no thread count set. That is not incidental. This solver pushes its work into BLAS calls, so its timings inherit whatever threading decision the installed library makes by default, and those defaults differ between libraries by more than the effects measured here. Accelerate exposes no thread control, so there was nothing to set; on a library that does, the same experiment can read differently. The residuals of the identity check below and the counts of §9.1 are machine-independent to the last digit or two of rounding, and are what the paper rests on.

Optional dependency. The comparison in §9.2 includes the reference C implementation where a wheel for it is installed; building it needs the C toolchain the package under study exists to avoid, so that row is omitted, with a printed note, on a host that lacks it. Every other row is unconditional.

References

[1] Daniel Arnström and Daniel Axehill. A unifying complexity certification framework for active-set methods for convex quadratic programming. *IEEE Transactions on Automatic Control*, 67(6):2758–2770, 2022.

[2] Daniel Arnström, Alberto Bemporad, and Daniel Axehill. A dual active-set solver for embedded quadratic programming using recursive LDL$^\top$ updates. *IEEE Transactions on Automatic Control*, 67(8):4362–4369, 2022.

[3] Christian Bischof and Charles Van Loan. The WY representation for products of Householder matrices. *SIAM Journal on Scientific and Statistical Computing*, 8(1):s2–s13, 1987.

[4] Natashia L. Boland. A dual-active-set algorithm for positive semi-definite quadratic programming. *Mathematical Programming*, 1997.

[5] Richard W. Cottle, Jong-Shi Pang, and Richard E. Stone. *The Linear Complementarity Problem*. Academic Press, Boston, 1992. Reprinted as SIAM Classics in Applied Mathematics 60, 2009.

[6] Anders Forsgren, Philip E. Gill, and Elizabeth Wong. Primal and dual active-set methods for convex quadratic programming. *Mathematical Programming*, 159(1–2), 2016.

[7] Donald Goldfarb and Ashok Idnani. Dual and primal-dual methods for solving strictly convex quadratic programs. In *Numerical Analysis*, volume 909 of *Lecture Notes in Mathematics*, pages 226–239. Springer, 1982.

[8] Donald Goldfarb and Ashok Idnani. A numerically stable dual method for solving strictly convex quadratic programs. *Mathematical Programming*, 27(1):1–33, 1983.

[9] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4th edition, 2013.

[10] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. SIAM, 2nd edition, 2002.

[11] Michael Hintermüller, Kazufumi Ito, and Karl Kunisch. The primal-dual active set strategy as a semismooth Newton method. *SIAM Journal on Optimization*, 13(3):865–888, 2002.

[12] Joaquim J. Júdice and Fernanda M. Pires. A block principal pivoting algorithm for large-scale strictly monotone linear complementarity problems. *Computers & Operations Research*, 21(5):587–596, 1994.

[13] H. Markowitz. Portfolio selection. *Journal of Finance*, 7(1):77–91, 1952.

[14] Katta G. Murty. *Linear Complementarity, Linear and Nonlinear Programming*, volume 3 of *Sigma Series in Applied Mathematics*. Heldermann Verlag, Berlin, 1988.

[15] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, New York, 2nd edition, 2006.

[16] Thomas Schmelzer and Enzo Montariol. `cvx-quadprog`: Goldfarb/Idnani dual quadratic programming in NumPy and SciPy, 2026. Software, version 0.4.2, https://doi.org/10.5281/zenodo.22147131.

[17] Thomas Schmelzer and Martin Stoll. Non-negative conjugate gradients. Preprint, https://arxiv.org/abs/2607.22121, 2026.

[18] Robert Schreiber and Charles Van Loan. A storage-efficient WY representation for products of Householder transformations. *SIAM Journal on Scientific and Statistical Computing*, 10(1):53–57, 1989.